

Southwest
Tech

Web Development Foundations

Week 5 - DOM: "Now It Connects"
Day 2: "DOM Input Feedback"

1

Southwest
Tech

Reconnect

Monday:

- ▶ selected page elements
- ▶ waited for a click
- ▶ ran a function
- ▶ changed visible text

Today:

- ▶ read user input
- ▶ check the value
- ▶ give clearer feedback

2

Southwest
Tech

From Monday's Button to Wednesday's Feedback

We use the same connection chain to make more helpful interactions.

1 button changes fixed text
The page is waiting.
Update message
The button changed the page.
Monday: One click updates one fixed message.

2 user types input
Your name:
Avery
Wednesday: The user enters information.

3 JavaScript checks value
Is the input empty?
Is it long enough?
Is it allowed?
JavaScript decides what should happen.
Thursday: The code checks the value and decides the result.

4 page shows feedback
Your name:
Avery
Nice to meet you, Avery!
The page shows helpful feedback to the user.

Same connection. More useful feedback.
Find elements. Listen for events. Run functions. Change what the user sees.

From Button To Input Feedback

3

Southwest
Tech

The Working Problem: Fixed Text Is Limited

A fixed message proves the connection works.
But users often need feedback based on what they do.

Examples:

- ▶ empty input needs a reminder
- ▶ typed input can personalize a message
- ▶ unclear feedback leaves users guessing

4

Southwest
Tech

Why Fixed Button Text Is Limited

The same message is not as helpful as a message that fits the user.

Fixed Button Text
The button always shows the same message.
Update message
Hello there!
same response
No matter what the user does, the message stays the same.

Input-Based Feedback
The message matches what the user types.
User types: Avery
Page response: Nice to meet you, Avery!
User types: Sam
Page response: Nice to meet you, Sam!
user-specific feedback
The message changes to fit what the user entered.

Feedback should respond to the user.
When the page responds to what the user does, it becomes more helpful and meaningful.

Fixed Text Is Limited

5

Southwest
Tech

What Counts as Success Today

- an input is selected
- the current `.value` is read
- empty input is handled
- feedback appears on the page
- selector mistakes are easier to spot

6

Inputs Make The Page Respond To The User

Input changes the interaction.

Without input:

- the page gives the same response every time

With input:

- the user provides a value
- JavaScript reads that value
- the page responds with feedback

7

Input Lets the Page Respond to the User

The page uses what the user types to give helpful feedback.

- 1 user provides value**
The user types information into the input field.
Your name:
- 2 JavaScript reads value**
JavaScript gets the value from the input field.
JS: JavaScript reads the value: "Avery"
- 3 page updates feedback**
The page shows a message based on the user's value.
Message:

The page responds with information from the user.
Input → JavaScript → Updated feedback

Input Response Concept

8

What We Will Use Today

- input element
- label
- `.value`
- `.trim()`
- empty string
- `return`
- feedback message
- selector check

9

Today's Input-Feedback Toolbox

The tools we need to read input and show helpful feedback.

input Let's the user type a value.	label Gives context for the input.	.value Gets the current value from input.	.trim() Removes extra spaces.
empty string Means nothing was entered.	return Stops the function early if needed.	feedback message Shows helpful information.	selector check Confirms the element exists before using it.

TODAY'S TOOLS

Small tools. Big impact.
Use these tools together to turn input into feedback the user can understand.

User → JS → Feedback

Input Feedback Toolbox

10

Selectors Must Match The HTML

The selector must match the page.

HTML:

```
<input id="nameInput">
```

JavaScript:

```
document.querySelector("#nameInput")
```

If the selector does not match, JavaScript may find `null`.

11

A JavaScript Selector Must Match the HTML

JavaScript finds elements by their id.

HTML The input element in the page. <pre><label for="nameInput">Your name:</label> <input type="text" id="nameInput" placeholder="Type your name..."></pre> Your name: Type your name... id = nameInput	JS JavaScript JavaScript uses a selector to find the element. <pre>const nameInput = document.querySelector("#nameInput");</pre> This selector looks for an element with id = nameInput
---	---

No match can mean null.
If the id in your HTML, and the selector in your JavaScript do not match exactly, `querySelector()` returns `null`.

- IDs are case-sensitive.
- Check spelling carefully.
- Check the # in your selector.

Selector Must Match HTML

12

Southwest Tech

Reading Input Values



Selecting the input finds the element.

- ▶ `.value` reads what the user typed.

Example:

```
const name =
  nameInput.value.trim();
```

Read it as:

"Get the current input text and remove extra spaces."

13

Southwest Tech

Selecting an Input Is Not Reading Its Value

Two different steps. Both are needed.

1 select input element

JavaScript finds the input element in the page by its id.

```
HTML
<input type="text" id="nameInput">
```

Your name:

```
const nameInput =
  document.querySelector("#nameInput");
```

2 read current text with .value

JavaScript reads the current text inside the input.

Your name:

```
nameInput.value = "Alex"
```

This gets the text the user has typed right now.

Select the element first. Read its value next.
First, get a reference to the input. Then, read the current text with `.value`.

Reading Input Values

14

Southwest Tech

Feedback Decision Branch



Good feedback often needs a condition.

If the input is empty:

- ▶ ask the user to enter something
- ▶ stop the function

Otherwise:

- ▶ use the input
- ▶ update the message

15

Southwest Tech

Feedback Is a Simple Decision Branch

One condition. Two possible responses.

```

graph LR
    User[User types in the input] --> Input[Your name: Alex]
    Input --> Decision{Is the input empty?}
    Decision -- YES --> Feedback[Please enter a name first.]
    Decision -- NO --> Welcome[Hello, Alex. Welcome to the page.]
  
```

Good feedback often starts with one condition.
Check something simple first. Give the user a clear, helpful message.

Feedback Decision Branch

16

Southwest Tech

Demo

"Input Feedback"

17

Southwest Tech

What to Watch For

- ▶ input, button, and message selected
- ▶ `.value` read after the click
- ▶ empty input checked
- ▶ feedback message updated
- ▶ click event still controls timing

18

Input Feedback
The page gives feedback based on what the user types.

Name
Type your name...
Create greeting

Message
Please enter a name first.

Name
Alex
Create greeting

Message
Hello, Alex. Welcome to the page.

Click reads the current input value. The page checks the value and shows the right message.

User types a name → JS Click reads the value → Page shows feedback

Demo: Input Feedback

19

Common DOM Connection Mistakes

- selector does not match the HTML
- script file is not linked
- function is defined but not connected to an event
- input element is selected but `.value` is not read
- console error is ignored

20

Common Beginner DOM Connection Mistakes
Use this checklist to find and fix the most common issues.

- 1 selector does not match HTML. Check if names, if vs ., and spelling. Even one character can break the connection.
- 2 script file is not linked. Make sure the `<script src="">` is in your HTML file.
- 3 function is not connected to an event. Add an event (like click) that calls your function.
- 4 input selected but `.value` not read. Select the element first. Then read the current text with `.value`.
- 5 console error ignored. Console messages show what went wrong and where to look.

Connection problems leave clues. Read carefully, check patterns, and test again.

Small checks lead to working pages!

Common DOM Connection Mistakes

21

Lab Refinement

Your goal:

- ▶ keep the JavaScript in a separate file
- ▶ make one interaction reliable
- ▶ add a second interaction or improve the first
- ▶ improve user feedback
- ▶ remove console errors

22

Evidence & Reflection

Evidence:

- ▶ updated HTML files
- ▶ CSS file
- ▶ separate JavaScript file
- ▶ at least one visible interaction
- ▶ no console errors

Reflection

- ▶ how page behavior changed your understanding

23

How To Read Next Week's Material

Required:

- ▶ read for debugging as a process
- ▶ notice console messages and common errors
- ▶ look for cause versus symptom

Reference:

- ▶ browser-specific tools are there when you need them
- ▶ -do not memorize every DevTools feature.

Before next time:

- ▶ Bring one broken or confusing behavior you would like to understand.

24



25

Closing Success Check

This week:

- ▶ JavaScript connected to HTML
- ▶ events triggered functions
- ▶ input values affected output
- ▶ the page responded to the user

Next:

- ▶ Debugging turns broken behavior into evidence.

26



27